

**CORSO DI LAUREA IN INFORMATICA
INSEGNAMENTO DI BASI DI DATI
ANNO ACCADEMICO 2025/2026**

Unina Multicloud

Gruppo OOB18:

- Alessandro Dell'Annunziata (DE1000169);
al.dellannunziata@studenti.unina.it
- Mario Corrado (DE1000320).
mario.corrado2@studenti.unina.it

Docente:

- prof.ssa Mara Sangiovanni;

INDICE

1. ANALISI DELLA TRACCIA.....	3
1.1 SCOPO DEL DOCUMENTO.....	3
1.2 DESCRIZIONE DEL PROGETTO.....	3
1.3 ENTITÀ PRINCIPALI.....	3
2. CLASS DIAGRAM.....	4
2.1 OSSERVAZIONI SUL CLASS DIAGRAM.....	5
2.1.1 Identificazione delle Classi e Scelte di Modellazione.....	5
2.1.2 Modellazione delle Gerarchie.....	5
2.1.3 Spiegazione dell'attributo risoluzione.....	6
2.1.4 Metodo di conteggio delle visualizzazioni.....	6
2.1.5 Doppia Associazione Utente-Elemento.....	6
2.1.6 Necessità della Classe di Associazione Fruizioni.....	7
2.1.7 Distribuzione Selettiva delle Associazioni sulle Playlist.....	7
3. MODELLO ER.....	7
3.1 OSSERVAZIONE SULL'ER.....	9
3.1.1 Entità deboli.....	9
3.1.2 Attributo multivalore.....	9
4. RISTRUTTURAZIONE DELLO SCHEMA CONCETTUALE.....	10
4.1 ELIMINAZIONE DEGLI ATTRIBUTI MULTIVALEORE (CATEGORIA).....	10
4.2 ANALISI DELLE RIDONDANZE E ATTRIBUTI DERIVATI.....	10
4.3 ELIMINAZIONE DELLE GERARCHIE.....	11
4.4 GESTIONE DELLE CLASSI DI ASSOCIAZIONE (FRUIZIONI).....	11
5. UML RISTRUTTURATO.....	12
6. DIZIONARIO DELLE CLASSI.....	13
7.DIZIONARIO DELLE ASSOCIAZIONI.....	16
8.DIZIONARIO DEI VINCOLI.....	18
9.SCHEMA LOGICO.....	20
10.SQL.....	22
10.1 CREATE TABLE.....	22
10.1.1 Utenti.....	22
10.1.2 ContenutiMultimediali.....	22
10.1.3 Fruizioni.....	22
10.1.4 Playlist.....	23
10.1.5 Playlist_pubblica.....	23
10.1.6 Playlist_condivisa.....	23
10.1.7 Playlist_privata.....	24
10.1.8 Categorie.....	24
10.1.9 appartiene_categoria.....	24

10.1.10 Accesso_o_Modifica.....	24
10.1.11 Contiene.....	25
10.1.12 Salva_o_visualizza.....	25
10.2 ALTER TABLE.....	25
10.2.1 Vincolo_Dominio_Email.....	25
10.2.2 Vincolo_Durata_Positiva.....	25
10.2.3 Vincolo_Gerarchia_Elemento.....	25
10.3 FUNCTION e TRIGGER.....	26
10.3.1 Vincolo_Derivazione_Views_Elemento.....	26
10.3.1 Vincolo_Derivazione_Views_Pubblica.....	26
10.3.1 Vincolo_Derivazione_Views_Pubblica.....	27

1. ANALISI DELLA TRACCIA

1.1 SCOPO DEL DOCUMENTO

Scopo del documento è esporre i requisiti, l'analisi e le soluzioni tecniche adottate relative al progetto Unina Multicloud.

1.2 DESCRIZIONE DEL PROGETTO

Il progetto "Unina Multicloud" prevede la realizzazione di una piattaforma riservata agli utenti dell'Università che consente di pubblicare e condividere contenuti multimediali audio/video.

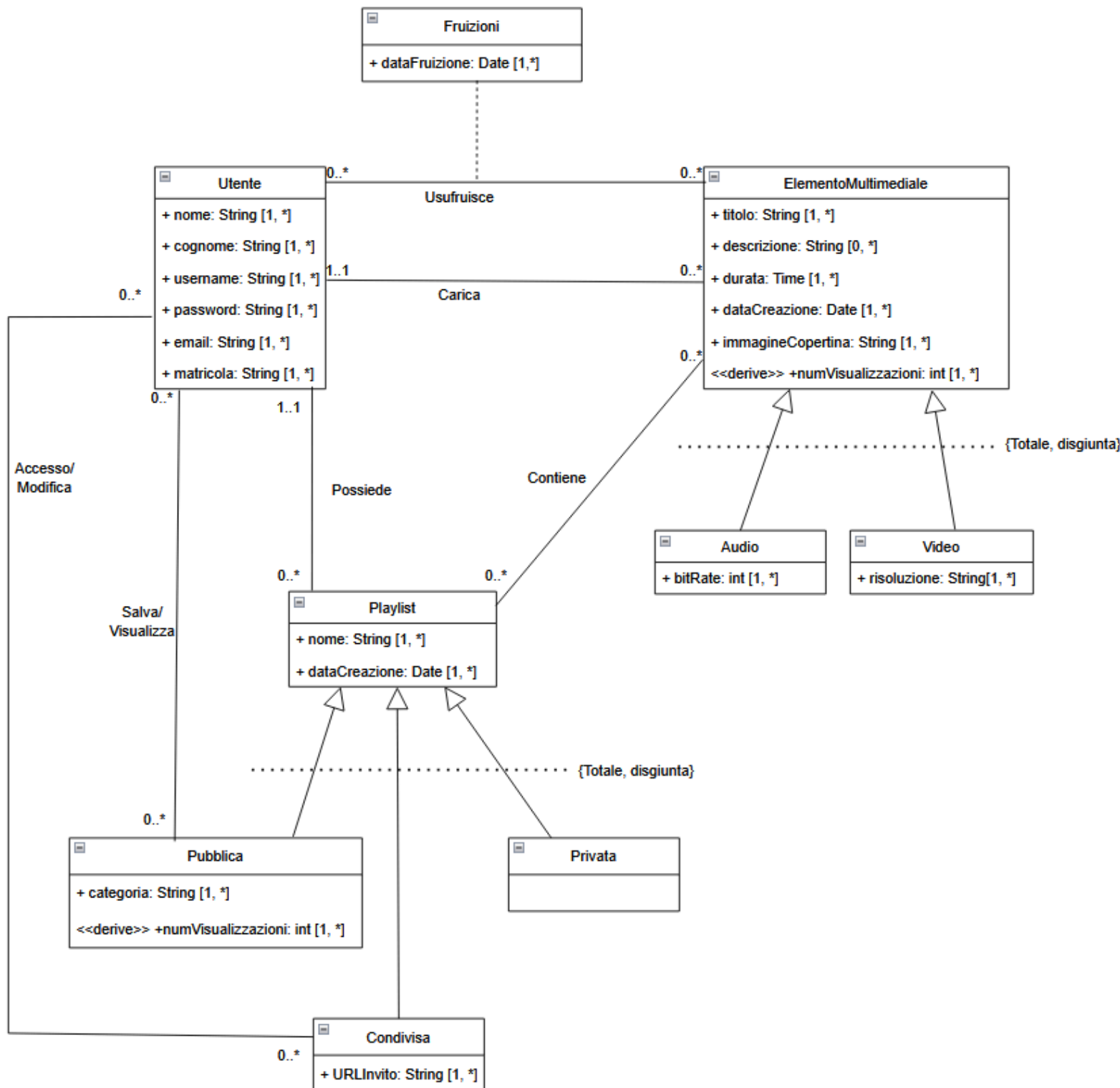
La base di dati, la cui progettazione verrà illustrata più avanti nel documento, sarà seguita da un applicativo scritto in Java.

1.3 ENTITÀ PRINCIPALI

Come prima cosa, abbiamo individuato le entità principali su cui avremmo dovuto lavorare ed il loro scopo effettivo.

Entità	Scopo	Descrizione
Utente	Conserva gli utenti nel DB	Questa è l'entità che descrive gli utenti.
ElementoMultimediale	Conserva gli elementi multimediali (audio/video)	Questa entità conserva gli attributi di un media qualsiasi
Playlist	Funge da raccolta di elementi multimediali	Può essere pubblica, condivisa o privata.

2. CLASS DIAGRAM



2.1 OSSERVAZIONI SUL CLASS DIAGRAM

2.1.1 Identificazione delle Classi e Scelte di Modellazione

Le classi individuate per soddisfare i requisiti della traccia sono le seguenti:

- **Utente:** Modella i soggetti accademici (studenti, docenti, personale) abilitati ad accedere alle funzionalità della piattaforma. Gli attributi associati (**nome**, **cognome**, **username**, **password**, **email**, **matricola**) definiscono il profilo informativo minimo richiesto per l'autenticazione e la tracciabilità.
- **ElementoMultimediale:** Classe astratta che fa da superclasse per i contenuti multimediali immessi nella piattaforma. Raccoglie tutti i dati comuni a qualsiasi tipologia di file riproducibile: **titolo**, **descrizione**, **durata**, **dataCreazione** e **immagineCopertina**.
- **Playlist:** La traccia delinea tre comportamenti d'uso e di visibilità profondamente differenti per le raccolte. Per evitare la frammentazione dei dati comuni (ogni playlist, a prescindere dal tipo, possiede un **nome** e una **data di creazione**), si è deciso di centralizzare questi attributi in una superclasse, demandando alle estensioni la gestione dei dettagli specifici e dei relativi diritti di accesso.

2.1.2 Modellazione delle Gerarchie

Le due gerarchie introdotte sono state definite in base a vincoli stringenti:

- La gerarchia su **ElementoMultimediale** è stata definita **totale** perché nel sistema non possono esistere elementi "generici": un file è o un audio o un video. È inoltre **disgiunta** perché un contenuto non può sovrapporsi contemporaneamente nei due formati (un file audio non può essere allo stesso tempo anche video e viceversa).
- La gerarchia su **Playlist** è anch'essa **totale** e **disgiunta**. Una playlist deve necessariamente nascere sotto una delle tre forme di

visibilità previste (**Privata**, **Pubblica**, **Condivisa**) e non può mutare la propria natura sovrapponendo i comportamenti (es. una playlist non può essere contemporaneamente privata e pubblica).

2.1.3 Spiegazione dell'attributo risoluzione

L'attributo "**risoluzione**" in Video è da intendersi come qualcosa del tipo: "1080p", "4K"... e per tanto è stato considerato come stringa.

2.1.4 Metodo di conteggio delle visualizzazioni

Entrambi gli attributi "**numVisualizzazioni**" in Pubblica ed in ElementoMultimediale sono derivati dal numero di occorrenze dell'associazione opportuna (salva/visualizza, usufrisce).

2.1.5 Doppia Associazione Utente-Elemento

Si è, inoltre, ritenuto fondamentale separare nettamente l'azione statica di pubblicazione dall'azione dinamica di fruizione.

- L'associazione **Carica (1:N)** stabilisce la paternità del file: un elemento deve avere uno e un solo autore (**1..1**), mentre un utente può non aver ancora caricato nulla (**0..***).
- L'associazione **Usufrisce (M:N)** ha una dinamica completamente diversa, in cui sia gli utenti che gli elementi partecipano con molteplicità aperta (**0..***), in quanto lo stesso video può essere visto da più utenti e un utente può fruire di più video.

2.1.6 Necessità della Classe di Associazione Fruizioni

Quando un utente guarda o ascolta un contenuto, questa interazione non deve perdersi nel nulla. Il database ha l'obbligo di storicizzare questo evento, altrimenti non avremmo i dati necessari per fare le statistiche, come ad esempio contare quante views ha fatto un video. Inoltre, l'attributo **dataFruizione** non poteva essere inserito in *Utente* (perché un utente vede più video in date diverse) né in *ElementoMultimediale* (perché un video è visto in date diverse da persone diverse). La soluzione è stata la creazione della classe di

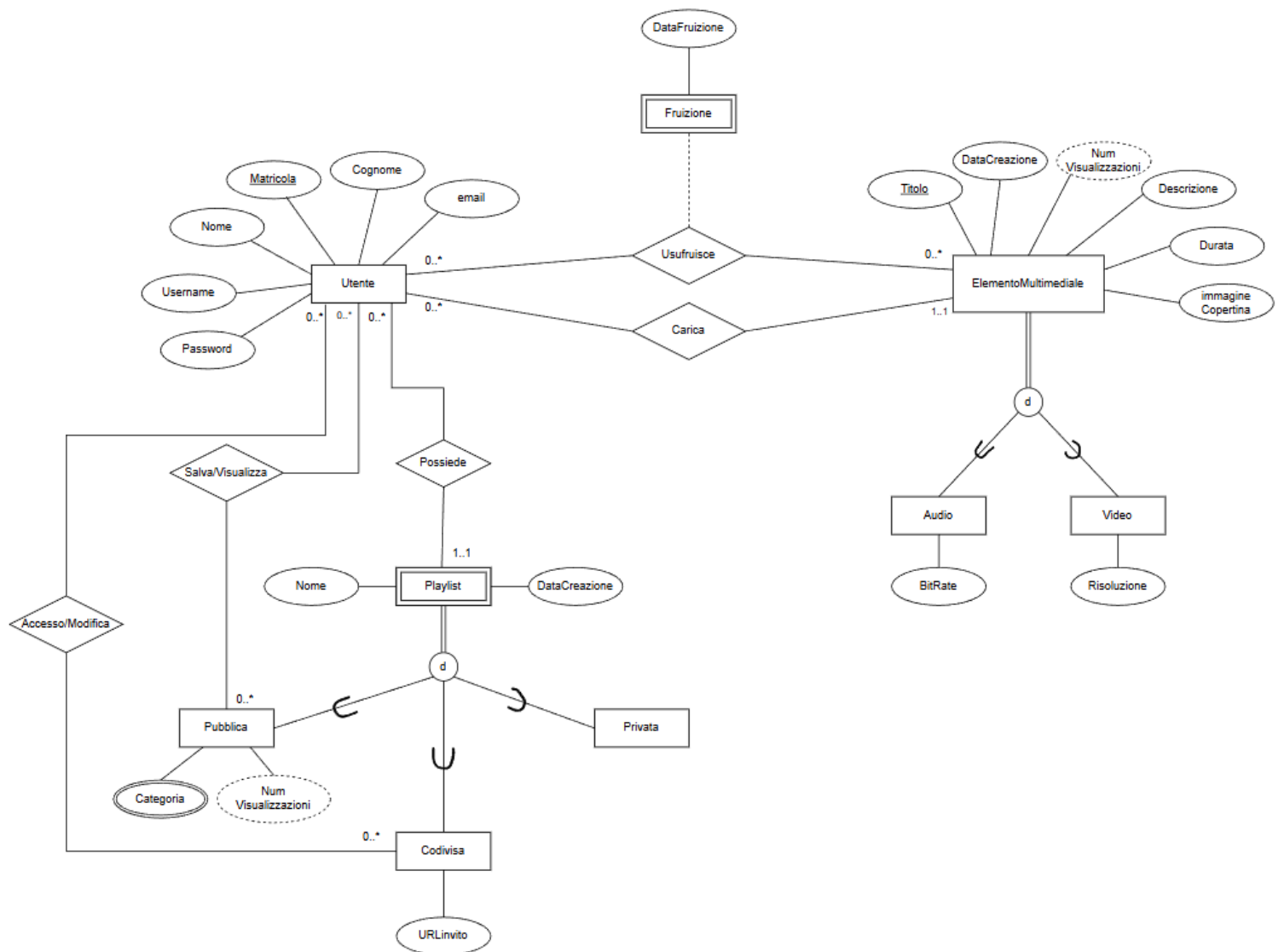
associazione **Fruizioni**, che interseca l'evento legandolo alla coppia specifica (Utente, Elemento).

2.1.7 Distribuzione Selettiva delle Associazioni sulle Playlist

Per implementare i requisiti di sicurezza e privacy, si è scelto di non far partire le associazioni di interazione dalla superclasse *Playlist*, bensì dalle singole sottoclassi:

- L'associazione **Salva/Visualizza** punta esclusivamente a **Pubblica**, poiché le playlist private o condivise non devono essere visibili o clonabili dalla generalità degli utenti.
- L'associazione **Accesso/Modifica** punta esclusivamente a **Condivisa**, limitando l'interazione collaborativa ai soli utenti che hanno ricevuto un esplicito invito, blindando il sistema da accessi non autorizzati.

3. MODELLO ER



3.1 OSSERVAZIONE SULL'ER

3.1.1 Entità deboli

Non avendo attributi identificativi propri, fruizione e playlist sono da considerarsi entità deboli.

3.1.2 Attributo multivalore

Essendo che una playlist pubblica può avere più categorie associate allo stesso tempo, abbiamo scelto di considerare e gestire categoria come un attributo multivalore

4. RISTRUTTURAZIONE DELLO SCHEMA CONCETTUALE

In questa fase abbiamo provveduto a rielaborare i costrutti del modello concettuale non direttamente compatibili con il modello relazionale, ottimizzando al contempo l'efficienza della base di dati.

4.1 ELIMINAZIONE DEGLI ATTRIBUTI MULTIVALORE (CATEGORIA)

Nello schema originario, la classe Pubblica possedeva l'attributo categoria (multivalore). Poiché il modello relazionale non ammette attributi strutturati, l'attributo è stato scorporato. È stata creata una nuova entità autonoma Categoria (contenente l'identificatore e il nome del genere), collegata alla classe Pubblica tramite una nuova associazione molti-a-molti (M:N).

4.2 ANALISI DELLE RIDONDANZE E ATTRIBUTI DERIVATI

L'attributo numVisualizzazioni (presente per gli elementi multimediali e le playlist pubbliche) è concettualmente un dato derivato, ottenibile contando il numero di tuple nelle rispettive associazioni di interazione. In fase di ristrutturazione, si è scelto di **mantenere il dato come ridondanza fisica** eliminando la derivazione.

4.3 ELIMINAZIONE DELLE GERARCHIE

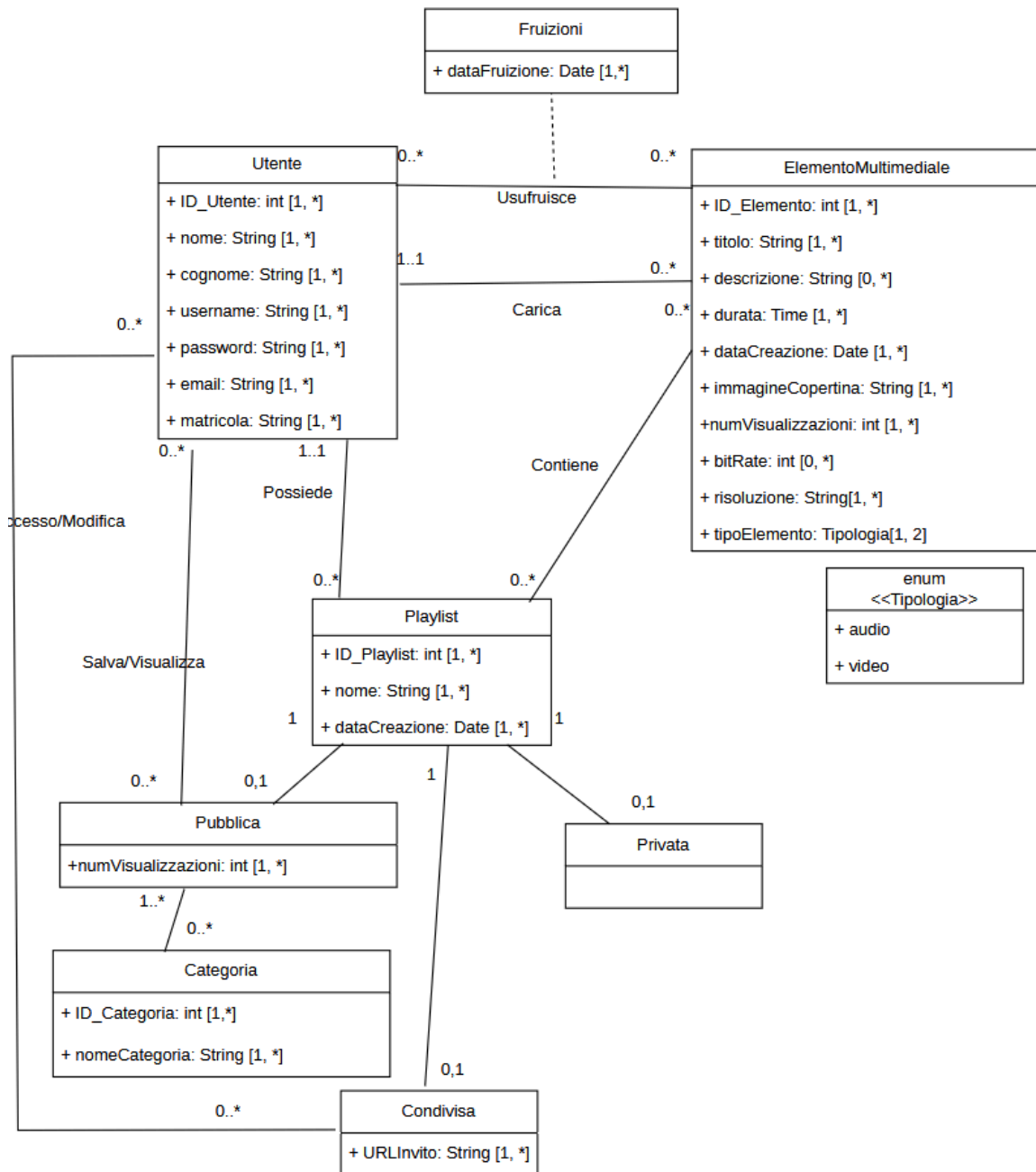
Il modello presentava due gerarchie, per le quali sono state adottate due strategie differenti in base all'analisi del carico e della struttura:

- **Gerarchia Audio/Video (Accorpamento nel Padre):** Le due sottoclassi possedevano un solo attributo specifico ciascuna (bitRate per Audio, risoluzione per Video) e non partecipavano ad associazioni esclusive. Il partizionamento in tabelle separate avrebbe generato continui e onerosi JOIN per ricostruire le informazioni di base (titolo, durata) durante le ricerche. Si è quindi optato per il collasso nel padre ElementoMultimediale, rendendo gli attributi specifici opzionali [0,1] e introducendo un attributo discriminante tipoElemento.
I controlli sui valori nulli sono demandati a vincoli di n-pla.
- **Gerarchia Playlist (Sostituzione con Associazioni):** A differenza della precedente, le sottoclassi Pubblica e Condivisa partecipano ad associazioni esclusive (rispettivamente con gli utenti). Un accorpamento nel padre avrebbe generato un'entità monolitica piena di valori nulli e con pericolose sovrapposizioni di permessi di accesso. La gerarchia UML è stata eliminata introducendo associazioni binarie 1 a 0..1 tra la ex-superclasse e le ex-sottoclassi, vincolate rigorosamente da regole interrelazionali di Totalità e Disgiunzione.

4.4 GESTIONE DELLE CLASSI DI ASSOCIAZIONE (FRUIZIONI)

L'associazione Usufruisce include la classe Fruizioni per tracciare la data di fruizione. In questa fase abbiamo mantenuto inalterato il costrutto grafico senza inserire chiavi esterne. La creazione della relazione ponte è stata posticipata alla fase di mapping logico, così da preservare la correttezza semantica dello schema concettuale.

5. UML RISTRUTTURATO



6. DIZIONARIO DELLE CLASSI

Nome Classe	Descrizione Testuale	Attributi, Tipo e Cardinalità
Utente	Utente registrato alla piattaforma universitaria.	ID_Utente: Int [1,*] : Identifica univocamente ciascuna istanza di Utente. Nome: String [1,*] : Nome associato all'utente. Cognome: String [1,*] : Cognome associato all'utente. Username: String [1,*] : Nome utente univoco associato all'utente. Password: String [1,*] : Usata per l'accesso all'account. email: String [1,*] : Email universitaria univoca associata all'utente. matricola: String [1,*] : Matricola universitaria univoca associata all'utente.

ElementoMultimediale	Classe che accorpa Audio e Video. Rappresenta i contenuti caricati.	ID_Elemento: Int [1,*] : Identifica univocamente ciascuna istanza di ElementoMultimediale. titolo: String [1,*] : Nome assegnato all'Elemento. descrizione: String [0,*] : paragrafo descrittivo assegnato all'elemento. durata: Time [1,*] : Tempo totale di riproduzione. dataCreazione: Date [1,*] : Giorno del caricamento. immagineCopertina: String [1,*]: Anteprima grafica (URL/Path). numVisualizzazioni: Int [1,*]: Conteggio totale del numero di fruizioni associate ad un ElementoMultimediale. tipoElemento: Enum [1,2] : Discriminante (Audio o Video). bitRate: Int [0,*] : Qualità audio in kbps. risoluzione: String [0,*] : Qualità video (es. 1080p).
-----------------------------	--	---

Playlist	Classe che rappresenta il contenitore logico di elementi.	ID_Playlist: Int [1,*] : Identifica univocamente ciascuna istanza di Playlist. Nome: String [1,*]: Titolo della raccolta. DataCreazione: Date [1,*]: Data della creazione della Playlist.
Pubblica	Estensione di Playlist, visibile a tutti.	numVisualizzazioni: Int [1,*] : Conteggio totale del numero di visualizzazioni.
Condivisa	Estensione di Playlist, accessibile su invito.	URLInvito: String [1,*] : Link univoco per accedere alla lista.
Privata	Estensione di Playlist, riservata al proprietario.	<i>(Nessun attributo proprio)</i>
Fruizione	Classe di associazione che registra l'interazione Utente-Elemento.	dataFruizione: date [1,*]: Giorno esatto della visualizzazione.
Categoria	Modella i generi o i raggruppamenti tematici (es. Pop, Rock, Documentario, Tutorato) a cui un contenuto multimediale pubblico può appartenere	ID_Categoria: int[1,*] : Identifica univocamente ciascuna istanza di Categoria.

		Categoria: String [1,*] : Genere di appartenenza (es. Rock).
--	--	---

7.DIZIONARIO DELLE ASSOCIAZIONI

Nome Associazione	Descrizione Testuale	Entità Coinvolte (Ruolo, Molteplicità)	Classe di Associazione collegata
Carica	Definisce quale utente ha pubblicato un determinato elemento.	Utente (Carica, 1..1) ElementoMultimediale (Viene caricato, 0..*)	-
Usufruisce	Traccia le visualizzazioni/ascolti degli elementi da parte degli utenti.	Utente (Guarda/ascolta, 0..*) ElementoMultimediale (Visionato/ascoltato, 0..*)	Fruizioni dataFruizione: Date [1,*]
Possiede	Associa un utente alla playlist di cui è creatore e proprietario.	Utente (Possiede, 0..*) Playlist (è posseduta, 1..1)	-

Contiene	Definisce gli elementi multimediali inseriti all'interno di una playlist.	Playlist (Contiene, 0..*) ElementoMultimediale (è contenuto, 0..*)	-
Salva/Visualizza	Permette agli utenti di seguire una playlist pubblica di altri.	Utente (Salva/visualizza, 0..*) Pubblica (è salvata/visualizzata, 0..*)	-
Accesso/Modifica	Assegna i permessi agli utenti scelti per una playlist condivisa.	Utente (Accede/Modifica, 0..*) Condivisa (è Vista/Modificata, 0..*)	-
Pubblica	Collega la classe padre Playlist alla sua estensione specifica.	Playlist (Super, 0..1) Pubblica (Sub, 1..1)	-
Privata	Collega la classe padre Playlist alla sua estensione specifica.	Playlist (Super, 0..1) Privata (Sub, 1..1)	-
Condivisa	Collega la classe padre Playlist alla sua estensione specifica.	Playlist (Super, 0..1) Condivisa (Sub, 1..1)	-

Appartiene_Categoria	Associa una playlist pubblica alle categorie/generi di appartenenza	Playlist(ha, 0..*) Categoria(appartiene, 0..*)	
-----------------------------	---	---	--

8.DIZIONARIO DEI VINCOLI

Nome	Tipo del vincolo	Descrizione del vincolo
Vincolo_Dominio_Email	Di dominio	L'attributo email nella classe Utente deve terminare obbligatoriamente con il dominio universitario. ("@unina.it")
Vincolo_Durata_Positiva	Di dominio	L'attributo durata in ElementoMultimediale deve essere > 0.
Vincolo_Email_Univoca	Intrarelazionale	L'attributo email nella classe Utente non ammette duplicati (UNIQUE).
Vincolo_Username_Univoco	Intrarelazionale	L'attributo username nella classe Utente non ammette duplicati (UNIQUE).

Vincolo_Gerarchia_Elemento	Di n-pla	Avendo accorpato Audio e Video. Se tipoElemento = 'audio', allora bitRate è NOT NULL e risoluzione è NULL. Se tipoElemento = 'video', allora la risoluzione è NOT NULL e bitRate è NULL.
Vincolo_Totalita_Playlist	Interrelazionale	Avendo trasformato la gerarchia Playlist in associazioni, ogni istanza della classe Playlist DEVE partecipare obbligatoriamente ad almeno una delle associazioni verso Privata, Pubblica o Condivisa.
Vincolo_Disgiunzione_Playlist	Interrelazionale	Un'istanza della classe Playlist NON PUÒ partecipare contemporaneamente a più di una associazione verso le classi Privata, Pubblica o Condivisa.
Vincolo_Derivazione_Views_Elemento	Interrelazione	L'attributo ridondante numVisualizzazioni in ElementoMultimediale si aggiorna ad ogni INSERT o DELETE nella tabella Fruizioni.
Vincolo_Derivazione_Views_Pubblica	Interrelazione	L'attributo ridondante numVisualizzazioni nella classe Pubblica si aggiorna ad ogni INSERT o DELETE nella tabella Salva_o_visualizza.

Nota: I due vincoli di totalità e disgiunzione playlist sono implementati nel DB come "trg_verifica_vincoli_playlist"

9.SCHEMA LOGICO

Entità	Attributi	Chiavi
Utente	ID_Utente matricola email password username cognome nome	PK
Fruizione	ID_Utente ID_Elemento dataFruizione	PK: {ID_Utente, ID_Elemento} FK1: ID_Utente → UTENTE FK2: ID_Elemento → ELEMENTO
ElementoMultimediale	ID_Elemento titolo descrizione durata dataCreazione immagineCopertina numVisualizzazioni bitRate risoluzione tipoElemento ID_Utente	PK FK: ID_Utente → UTENTE
Salva_o_Visualizza	ID_Utente ID_Playlist	PK: {ID_Utente, ID_Playlist} FK1: ID_Utente → UTENTE FK2: ID_Playlist → PUBBLICA
Accesso_o_Modifica	ID_Utente ID_Playlist	PK: {ID_Utente, ID_Playlist} FK1: ID_Utente → UTENTE

		FK2: ID_Playlist → CONDIVISA
Contiene	ID_Elemento ID_Playlist	PK: {ID_Playlist, ID_Elemento} FK1: ID_Playlist → PLAYLIST FK2: ID_Elemento → ELEMENTO
Playlist	ID_Playlist nome dataCreazione ID_Utente	PK FK: ID_Utente → UTENTE
Pubblica	ID_Playlist categoria numVisualizzazioni	PK, FK: ID_Playlist → PLAYLIST
Condivisa	ID_Playlist URLInvito	PK, FK: ID_Playlist → PLAYLIST
Privata	ID_Playlist	PK, FK: ID_Playlist → PLAYLIST
Categoria	ID_Categoria NomeCategoria	PK
Appartiene_Categoria	ID_Categoria ID_Playlist	PK: {ID_Playlist, ID_Categoria} FK1: ID_Playlist → PUBBLICA FK2: ID_Categoria → CATEGORIA

10.SQL

10.1 CREATE TABLE

10.1.1 Utenti

```
CREATE TABLE Utenti(  
    ID_Utente SERIAL PRIMARY KEY,  
    nome VARCHAR(50) NOT NULL,  
    cognome VARCHAR(50) NOT NULL,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    psswrđ VARCHAR(50) NOT NULL,  
    email VARCHAR(50) UNIQUE NOT NULL,  
    matricola CHAR(9) UNIQUE  
);
```

10.1.2 ContenutiMultimediali

```
CREATE TABLE ContenutiMultimediali(  
    ID_Elemento SERIAL PRIMARY KEY,  
    titolo VARCHAR(50) NOT NULL,  
    descrizione VARCHAR(500),  
    durata BIGINT NOT NULL,  
    dataCreazione DATE NOT NULL,  
    immagineCopertina VARCHAR(500),  
    numVisualizzazioni BIGINT DEFAULT 0,  
    tipoElemento CHAR(5) NOT NULL,  
    bitRate SMALLINT,  
    risoluzione VARCHAR(10),  
    ID_Utente INT NOT NULL,  
    FOREIGN KEY(ID_Utente) REFERENCES Utenti(ID_Utente)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

10.1.3 Fruizioni

```
CREATE TABLE Fruizioni(  
  
    ID_Utente INT NOT NULL,  
    ID_Elemento INT NOT NULL,  
    dataFruizione TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (ID_Utente) REFERENCES Utenti(ID_Utente)  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Elemento) REFERENCES ContenutiMultimediali(ID_Elemento)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

10.1.4 Playlist

CREATE TABLE Playlist(

ID_Playlist SERIAL PRIMARY KEY,
nome VARCHAR(50),
dataCreazione TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
ID_Utente INT,
FOREIGN KEY (ID_Utente) REFERENCES Utenti (ID_Utente)
ON DELETE CASCADE
ON UPDATE CASCADE

);

10.1.5 Playlist_pubblica

CREATE TABLE Playlist_pubblica(

ID_Playlist SERIAL PRIMARY KEY,
categoria VARCHAR(50),
numVisualizzazioni BIGINT DEFAULT 0,
ID_Utente INT,
FOREIGN KEY (ID_Utente) REFERENCES Utenti (ID_Utente)
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (ID_Playlist) REFERENCES Playlist (ID_Playlist)
ON DELETE CASCADE
ON UPDATE CASCADE

);

10.1.6 Playlist_condivisa

CREATE TABLE Playlist_Condivisa(

ID_Playlist SERIAL PRIMARY KEY,
URL_Invito VARCHAR(200) NOT NULL,
ID_Utente INT,
FOREIGN KEY (ID_Utente) REFERENCES Utenti (ID_Utente)
ON DELETE CASCADE
ON UPDATE CASCADE,
FOREIGN KEY (ID_Playlist) REFERENCES Playlist (ID_Playlist)
ON DELETE CASCADE
ON UPDATE CASCADE

);

10.1.7 Playlist_privata

```
CREATE TABLE Playlist_Privata(  
  
    ID_Playlist SERIAL PRIMARY KEY,  
    ID_Utente INT,  
    FOREIGN KEY (ID_Utente) REFERENCES Utenti (ID_Utente)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Playlist) REFERENCES Playlist (ID_Playlist)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
  
);
```

10.1.8 Categorie

```
CREATE TABLE Categorie(  
    ID_Categoria SERIAL PRIMARY KEY,  
    nomeCategoria VARCHAR(50)  
  
);
```

10.1.9 appartiene_categoria

```
CREATE TABLE appartiene_categoria(  
  
    ID_Categoria INT,  
    ID_Playlist INT,  
    FOREIGN KEY (ID_Categoria) REFERENCES Categorie (ID_Categoria)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Playlist) REFERENCES Playlist_Pubblica (ID_Playlist)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    PRIMARY KEY (ID_Categoria, ID_Playlist)  
  
);
```

10.1.10 Accesso_o_Modifica

```
CREATE TABLE Accesso_o_Modifica(  
    ID_Utente INT,  
    ID_Playlist INT,  
    FOREIGN KEY (ID_Utente) REFERENCES Utenti(ID_Utente)  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Playlist) REFERENCES Playlist_Condivisa(ID_Playlist)  
    ON UPDATE CASCADE  
  
);
```

10.1.11 Contiene

```
CREATE TABLE Contiene(  
    ID_Elemento INT,  
    ID_Playlist INT,  
    FOREIGN KEY (ID_Elemento) REFERENCES ContenutiMultimediali (ID_Elemento)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Playlist) REFERENCES Playlist (ID_PLaylist)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
);
```

10.1.12 Salva_o_visualizza

```
CREATE TABLE salva_o_Visualizza(  
    ID_Utente INT,  
    ID_Playlist INT,  
    FOREIGN KEY (ID_Utente) REFERENCES Utenti(ID_Utente)  
    ON UPDATE CASCADE,  
    FOREIGN KEY (ID_Playlist) REFERENCES Playlist_pubblica(ID_PLaylist)  
    ON UPDATE CASCADE  
);
```

10.2 ALTER TABLE

10.2.1 Vincolo_Dominio_Email

```
ALTER TABLE Utenti  
ADD CONSTRAINT Vincolo_Dominio_Email CHECK(utenti.email LIKE '%@unina.it'); /*da  
cambiare nella documentazione ad intrarelazionale*/
```

10.2.2 Vincolo_Durata_Positiva

```
ALTER TABLE ContenutiMultimediali  
ADD CONSTRAINT Vincolo_Durata_Positiva CHECK(durata > 0);
```

10.2.3 Vincolo_Gerarchia_Elemento

```
ALTER TABLE ContenutiMultimediali  
ADD CONSTRAINT Vincolo_Gerarchia_Elemento CHECK(tipoelemento = 'video' AND  
risoluzione IS NOT NULL AND bitrate IS NULL OR  
tipoelemento = 'audio' AND bitrate IS NOT NULL AND risoluzione IS NULL);
```

10.3 FUNCTION e TRIGGER

10.3.1 Vincolo_Derivazione_Views_Elemento

```
CREATE OR REPLACE FUNCTION aggiorna_num_visualizzazioni()
RETURNS TRIGGER AS $$
BEGIN
    IF(TG_OP = 'INSERT') THEN
        UPDATE contenutimultimediali
        SET Numvisualizzazioni = numvisualizzazioni + 1
        WHERE ID_Elemento = OLD.ID_Elemento;
        RETURN NEW;
    ELSEIF(TG_OP = 'DELETE') THEN
        UPDATE contenutimultimediali
        SET Numvisualizzazioni = numvisualizzazioni - 1
        WHERE ID_Elemento = OLD.ID_Elemento;
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER vincolo_derivazione_views_elemento
AFTER INSERT OR DELETE ON Fruizioni
FOR EACH ROW
EXECUTE FUNCTION aggiorna_num_visualizzazioni();
```

10.3.1 Vincolo_Derivazione_Views_Pubblica

```
CREATE OR REPLACE FUNCTION deriva_views_playlist_pubblica()
RETURNS TRIGGER AS $$
BEGIN
    IF(TG_OP = 'INSERT') THEN
        UPDATE playlist_Pubblica
        SET numvisualizzazioni = numvisualizzazioni + 1
        WHERE ID_Playlist = NEW.ID_Playlist;
        RETURN NEW;
    ELSEIF(TG_OP = 'DELETE') THEN
        UPDATE playlist_Pubblica
        SET numvisualizzazioni = numvisualizzazioni - 1
        WHERE ID_Playlist = OLD.ID_Playlist;
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER vincolo_derivazione_views_pubblica
AFTER INSERT OR DELETE ON salva_o_visualizza
FOR EACH ROW
EXECUTE FUNCTION deriva_views_playlist_pubblica();
```

10.3.1 Vincolo_Derivazione_Views_Pubblica

```
CREATE OR REPLACE FUNCTION verifica_vincoli_playlist()
RETURNS TRIGGER AS $$
DECLARE
    conteggio INT := 0;
BEGIN
    -- Contiamo in quante tabelle figlie appare l'ID della playlist attuale
    SELECT
        (SELECT COUNT(*) FROM Playlist_Privata WHERE ID_Playlist = NEW.ID_Playlist) +
        (SELECT COUNT(*) FROM Playlist_Pubblica WHERE ID_Playlist = NEW.ID_Playlist) +
        (SELECT COUNT(*) FROM Playlist_Condivisa WHERE ID_Playlist = NEW.ID_Playlist)
    INTO conteggio;

    -- Vincolo di Totalità: conteggio non può essere 0
    IF conteggio = 0 THEN
        RAISE EXCEPTION 'Violazione Vincolo_Totalita_Playlist: La playlist % deve essere
associata a Privata, Pubblica o Condivisa.', NEW.ID_Playlist;
    -- Vincolo di Disgiunzione: conteggio non può essere maggiore di 1
    ELSIF conteggio > 1 THEN
        RAISE EXCEPTION 'Violazione Vincolo_Disgiunzione_Playlist: La playlist % non può
appartenere a più di una categoria contemporaneamente.', NEW.ID_Playlist;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE CONSTRAINT TRIGGER trg_verifica_vincoli_playlist
AFTER INSERT OR UPDATE ON Playlist
DEFERRABLE INITIALLY DEFERRED -- Fondamentale: controlla solo alla fine della
transazione
FOR EACH ROW
EXECUTE FUNCTION verifica_vincoli_playlist();
```

La clausola DEFERRABLE INITIALLY DEFERRED applicata al trigger trg_verifica_vincoli_playlist è fondamentale per gestire il problema della creazione di playlist specializzate ed il collegamento con Playlist (generica).

Senza quella clausola il TRIGGER verrebbe controllato immediatamente dopo un INSERT o un UPDATE in playlist; dato che al momento della INSERT in playlist non c'è ancora la riga corrispondente nella tabella specializzata, il conteggio nella FUNCTION sarà sempre 0.

Al contrario se si crea prima la playlist specializzata prima di quella generica, non si potrebbe collegare la Foreign Key dato che la Playlist generica ancora non esiste.

Il DEFERRABLE INITIALLY DEFERRED ci permette di creare la playlist generica, mettere in pausa il controllo del trigger finché non si completa l'operazione di creazione della playlist specializzata.